Chapter II. MATHEMATICAL FOUNDATION & DEEP DIVE

2.1. Markov Decision Process (MDP) and Objective Function

To enable an agent to learn how to balance a pole, the problem is formulated as a Markov Decision Process (MDP). At each timestep t, the interaction unfolds as follows:

- The agent observes the current state s_t.
- The agent takes an action a_t based on its policy network $\pi_\theta(a_t mid s_t)$.
- The environment returns a reward r_t and transitions the agent to the next state s_{t+1}.

This sequence of interactions forms a trajectory (episode) denoted as $\tau = (s_0, a_0, r_1, \dots, a_T, r_{T+1})$.

Intuition - Mapping to CartPole-v1 Environment:

To ground these mathematical variables in our code implementation:

- State s_t: A 4-dimensional array ([cart position, cart velocity, pole angle, pole angular velocity]).
- Action a_t: A discrete space with 2 values (0: Push left, 1: Push right).
- Reward r_t: Yields $+1$ for every frame the pole remains upright.
- Policy π_θ: A Multi-Layer Perceptron (MLP) neural network with a Softmax activation at the output, calculating the probability percentage of pushing left versus pushing right.

The ultimate goal of the agent is to optimize the neural network's parameters (weights) θ to maximize the Expected Total Reward, denoted as $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

The "The Optimization Challenge (Why not use standard derivatives?):

We cannot directly compute the derivative of $J(\theta)$ because the reward $R(\tau)$ generated by the environment is a "without explicit analytical form". We do not have a differentiable mathematical equation for the physics of the CartPole environment. Furthermore, rewards are delayed—if the pole falls at second 10, it does not necessarily mean the action at second 10 was wrong; the fatal mistake might have occurred at second 2.

2.2. Methodology: Policy Gradient Theorem

To bypass the environment's "lack of explicit analytical form, mathematicians employ a classic calculus technique known as the Log-derivative Trick.

2.2.1. The Log-derivative Trick and Practical Significance

In calculus, the derivative of lnx is $\frac{1}{x}$. Applying the chain rule yields:

$$\nabla_\theta ln\pi_\theta(amids) = \frac{\nabla_\theta \pi_\theta(amids)}{\pi_\theta(amids)} \Rightarrow \nabla_\theta \pi_\theta(amids) = \pi_\theta(amids)\nabla_\theta ln\pi_\theta(amids)$$

Logic Defense: Why transform into Logarithms?

Using the logarithm measures relative change rather than absolute change.

For example: If a good action initially has a very small probability (e.g., 1%), adding an absolute 1% (making it 2%) effectively doubles its likelihood (100% relative increase). Conversely, if an action already has a 90% probability, adding 1% is insignificant. The logarithmic derivative $\nabla ln\pi$ naturally captures this percentage change. It forces the neural network to strongly update good actions that are currently "currently having low probability" (low probability) while avoiding over-updating actions that are already highly probable.

2.2.2. Proof of the Policy Gradient Theorem

Using the Log-derivative trick, we can reformulate the objective function $J(\theta)$. We start by writing the expectation as a sum over probabilities:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau \mid \theta) R(\tau) = \sum_\tau \nabla_\theta P(\tau \mid \theta) R(\tau)$$

Applying the Log-derivative Trick to $P(\tau \mid \theta)$:

$$\nabla_\theta J(\theta) = \sum_\tau P(\tau \mid \theta) \nabla_\theta lnP(\tau \mid \theta) R(\tau)$$

Converting the sum back into an expectation form ($\mathbb{E}$):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[\nabla_\theta lnP(\tau \mid \theta) R(\tau)]$$

The probability of a trajectory $P(\tau \mid \theta)$ is the product of the initial state distribution, the environment's transition dynamics, and the policy's probabilities. When taking the logarithm, this product becomes a sum. When differentiating with respect to θ, all terms belonging to the environment (which do not contain θ) equate to zero and vanish entirely. Thus, we arrive at the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[\sum_{t=0}^{T} \nabla_\theta ln\pi_\theta(a_t mids_t) R(\tau)]$$

Intuition: This equation is an RL important result. It states that we do not need to understand the physical laws of the CartPole environment. We simply take the log-probability of the action predicted by the neural network, multiply it by the achieved total reward $R(\tau)$, and update the weights. High rewards reinforce the actions, while low rewards penalize them.

2.3. Deep Dive: REINFORCE Algorithm & Variance Analysis

Based on the proven Policy Gradient Theorem, the REINFORCE algorithm was created to translate this theory into practical weight updates. Its power lies in combining a stochastic policy with three core mechanisms.

2.3.1. Gradient Ascent and the "Implementation Detail"

In traditional Machine Learning (e.g., image classification), we use Gradient Descent to find the lowest point of a Loss Function (like descending into a minimum).

Conversely, in RL, our objective $J(\theta)$ is the Total Reward. Therefore, we must use Gradient Ascent to climb towards the highest reward:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Implementation Detail: By default, PyTorch is designed only for Gradient Descent (minimization). To "trick" PyTorch into performing Gradient Ascent (maximization), our codebase (reinforce.py) appends a negative sign to the Loss: policy_loss.append(-log_prob * G_t). Minimizing a negative value is mathematically equivalent to maximizing its positive counterpart.

2.3.2. Causality and Reward-to-go (G_{t})

According to the original theorem, the gradient is multiplied by $R(\tau)$ (the total reward of the entire episode). However, this exposes a logical flaw: A current action cannot change the rewards already received in the past.

Overcoming the Delay: Suppose the agent survives 100 steps. At step $t = 99$, the agent makes a fatal mistake and the pole falls. If we use the total $R(\tau)$, the bad action at step 99 is still multiplied by the massive reward accumulated from steps 1 to 98. This is illogical and confuses the network.

To resolve this, REINFORCE applies Causality, replacing the total $R(\tau)$ with the Reward-to-go G_t—calculating only the rewards from timestep t onwards:

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

With this refinement, if an action at time t leads to immediate termination, G_t will be extremely small, naturally reducing the probability of that poor action.

2.3.3. Variance Reduction via Baseline Subtraction

A specific trait of CartPole-v1 is that rewards are strictly positive ($+1$ for every surviving step). Consequently, the algorithm will technically "praise" (increase the probability of) all actions, varying only by the magnitude of the praise.

Numerical Intuition:

- Trajectory A performs excellently: $G_t = 500$ (Grade A+)
- Trajectory B performs poorly: $G_t = 10$ (Grade F)

Without a baseline, the system still adds weights for Trajectory B (since $10 > 0$). This causes massive variance in the gradient, leading to slow and erratic learning.

The Solution: We introduce a baseline $b(s_t)$, such as an average score of 100. The algorithm subtracts 100 from G_t:

- Trajectory A: $500 - 100 = +400$ → Reward (Strongly increase probability).
- Trajectory B: $10 - 100 = -90$ → Penalize (Decrease probability of the bad action).

Mathematical Proof: The Baseline is Unbiased

Does arbitrarily subtracting the mean (like returns = returns - returns.mean() in our code) violate the original math? The answer is NO. Let us examine the baseline term inside the gradient expectation:

$$\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta ln \pi_\theta(a_t mids_t) b(s_t)] = \sum_{a_t} \pi_\theta(a_t mids_t) \frac{\nabla_\theta \pi_\theta(a_t mids_t)}{\pi_\theta(a_t mids_t)} b(s_t)$$

Canceling out π_θ and pulling the baseline constant out of the summation:

$$= b(s_t) \nabla_\theta \sum_{a_t} \pi_\theta(a_t mids_t)$$

By the law of probability, the sum of all action probabilities at a given state always equals 1 ($\sum \pi = 1$), and the derivative of a constant is 0:

$$= b(s_t) \nabla_\theta (1) = b(s_t) \times 0 = 0$$

Conclusion: This elegant proof confirms that the baseline technique successfully eliminates variance (noise) and calibrates what is "good" versus "bad" without altering the true expectation (the unbiased direction) of the Policy Gradient. This is the final piece that makes the REINFORCE algorithm stable in practice.

2.4. Conclusion

Through Chapter 2, we have established a mathematical basis for solving the CartPole-v1 problem. Starting from modeling the environment as a Markov Decision Process (MDP), we proved the Policy Gradient Theorem through the Log-derivative Trick, transforming a optimization without explicit analytical form problem into a measurable expectation-based objective.

More importantly, through the deep dive into the REINFORCE algorithm, we bridged the gap between theory and practice by addressing the delayed-reward problem with causality and Reward-to-go G_t, and the high-variance problem with Baseline Subtraction. This tight connection between mathematical reasoning and implementation logic becomes the core premise for building the neural network and training pipeline in Chapter 3.